

Manuale di funzionamento Sistemi operativi

Francesco Cavedon (VR456164) e Paolo Gilberti (VR471880)

Settembre 2023

1 Applicazione

L'applicazione implementa il gioco Forza 4 in ambiente UNIX/Linux ed è scritta in linguaggio C, utilizzando le principali system call per la comunicazione tra processi.

2 Struttura

L'applicazione è composta da due eseguibili e, dal punto di vista del codice sorgente, è strutturata nel seguente modo:

- Un file dedicato al server, cioè la parte che si occupa della comunicazione tra processi e arbitra una partita;
- Un file dedicato al client, cioè la parte che gestisce il turno di un giocatore;
- Una serie di librerie per la definizione di costanti, strutture ed unioni e l'implementazione di funzioni.

2.1 Server

```
./F4Server <righe> <colonne> <gettone1> <gettone2>
```

Il server verifica innanzitutto che i parametri nella riga di comando siano corretti e gestisce la parte di comunicazione tra processi, creando la memoria condivisa, la coda di messaggi e i semafori. A questo punto, controlla che i giocatori si siano collegati e avviene uno scambio di messaggi. Dopo aver ricevuto le informazioni di cui ha bisogno, inizializza la griglia e definisce il time out entro il quale ogni giocatore deve completare il proprio turno. Una volta che la partita è iniziata, il server ha il compito di verificare, dopo ogni turno, se un giocatore ha vinto o se la griglia è piena. Al termine della partita, comunica ai giocatori il risultato e rimuove la memoria condivisa, la coda di messaggi e i semafori.

2.2 Client

```
./F4Client <giocatore> [*]
```

Il client verifica prima di tutto che i parametri nella riga di comando siano corretti e acquisisce gli identificativi della memoria condivisa, della coda di messaggi e dei semafori, precedentemente creati dal server. A questo punto, comunica al server di essersi collegato e riceve il gettone da utilizzare nella partita: tutte queste operazioni avvengono tramite uno scambio di messaggi. Il gioco inizia quando anche l'altro giocatore si è connesso. Le operazioni che vengono eseguite in un turno sono fondamentalmente quattro:

1. La griglia viene stampata a video;
2. Il giocatore sceglie la colonna in cui vuole inserire il proprio gettone;
3. La griglia viene aggiornata;
4. La griglia viene nuovamente stampata a video.

In questo modo, i giocatori inseriscono a turno il loro gettone e la partita termina quando il server comunica che qualcuno ha vinto o che la griglia è piena.

2.3 Librerie

La maggior parte delle funzioni sono state suddivise in librerie per semplificare sia le operazioni che riguardano la gestione di una partita sia le operazioni che coinvolgono la coda di messaggi, la memoria condivisa e i semafori.

Inoltre, è stato creato un modulo dedicato solamente all'inclusione di altre librerie e alla definizione di costanti e strutture.

3 Comunicazione tra processi

3.1 Coda di messaggi

La coda di messaggi viene utilizzata per lo scambio di alcune informazioni tra server e client nel momento in cui un giocatore si collega.

Un client sfrutta la coda di messaggi per comunicare al server il proprio pid, il nome del giocatore che si è collegato e l'intenzione di sfidare o meno un giocatore automatico (bot).

Diversamente, il server utilizza la coda di messaggi per comunicare a un client il proprio pid e il gettone che il giocatore deve utilizzare nella partita.

3.2 Memoria condivisa

La memoria condivisa viene utilizzata per memorizzare le informazioni di cui server e client hanno bisogno per gestire una partita: le dimensioni e il contenuto della griglia, il gettone del giocatore che ha appena completato il proprio turno, la durata del time out e un flag che indica se un giocatore ha inserito il proprio gettone in tempo. Tutti questi dati fanno parte di una struttura e la loro condivisione permette di semplificare l'inserimento di un gettone nella griglia e le operazioni di verifica eseguite dal server.

3.3 Semafori

Quando si svolge una partita, server e client devono entrare in una zona di memoria condivisa. Per questo motivo, è necessario che si sincronizzino ed accedano ai dati uno alla volta. Tutto ciò è stato implementato con dei semafori binari.

3.3.1 Giocatore vs. giocatore

Nel caso di una partita tra due giocatori sono necessari quattro semafori binari. Di seguito è riportato il modo in cui il server e i due client si sincronizzano per entrare nella zona di memoria condivisa:

	Server	Client
S0 = 0		
S1 = 0	while(1)	while(1)
S2 = 1	V(S0)	P(S2)
S3 = 1	P(S1)	P(S0)
	Operazioni	P(S3)
	V(S3)	V(S2)
		Operazioni
		V(S1)

3.3.2 Giocatore vs. bot

Nel caso di una partita tra un giocatore e un bot sono necessari due semafori binari. Di seguito è riportato il modo in cui il server e il client si sincronizzano per entrare nella zona di memoria condivisa:

	Server	Client
S4 = 0	while(1)	while(1)
S5 = 1	P(S4)	P(S5)
	Operazioni	Operazioni
	V(S5)	V(S4)

3.4 Segnali

3.4.1 Server

Durante la sua esecuzione, il server può gestire tre tipi di segnali:

- **SIGINT** viene ricevuto quando si premono i tasti CTRL-C. La prima pressione informa che una seconda pressione comporta la terminazione del gioco. Diversamente, con la seconda pressione, il server comunica ai giocatori che il gioco è stato terminato dall'esterno, rimuove la coda di messaggi, la memoria condivisa e i semafori e termina la sua esecuzione;
- **SIGTERM** viene ricevuto quando un giocatore abbandona la partita. Il server riceve un messaggio dal giocatore che ha abbandonato la partita, comunica all'altro che ha vinto a tavolino e termina la sua esecuzione;
- **SIGALRM** viene ricevuto quando non si verifica la seconda pressione dei tasti CTRL-C. La variabile che conta quante volte i tasti sono stati premuti viene azzerata perché la seconda pressione non è arrivata in tempo.

3.4.2 Client

Durante la sua esecuzione, il client può gestire cinque tipi di segnali:

- **SIGINT** viene ricevuto quando si premono i tasti CTRL-C. Il giocatore comunica al server che ha abbandonato la partita e termina la sua esecuzione;
- **SIGTERM** viene ricevuto quando l'altro giocatore abbandona la partita. Il giocatore viene a sapere di aver vinto a tavolino e termina la sua esecuzione;
- **SIGUSR1** viene ricevuto quando un giocatore ha vinto o quando la griglia è piena. Il giocatore viene a sapere se ha vinto o se la partita è finita in pareggio e termina la sua esecuzione;
- **SIGUSR2** viene ricevuto quando la partita è terminata dall'esterno. Il giocatore viene a sapere che la partita è stata terminata dall'esterno e termina la sua esecuzione;
- **SIGALRM** viene ricevuto quando scade il time out entro il quale un giocatore deve completare il proprio turno.

4 Note

4.1 Esecuzione del client

Per evitare problemi con il carattere speciale `*` nella riga di comando, il client viene eseguito specificando il parametro `\*`.

4.2 Chiavi di creazione

Per fare in modo che server e client utilizzino la stessa coda di messaggi, la stessa memoria condivisa e gli stessi semafori, le chiavi di creazione vengono generate con la funzione `ftok`.

4.3 Scadenza del time out

Se un giocatore non completa il proprio turno prima della scadenza del time out, la partita prosegue e inizia il turno dell'altro giocatore.

4.4 Esecuzione del bot

Nel caso di una partita tra un giocatore e un bot, il server deve duplicarsi ed eseguire un caso particolare di client, che gioca un turno inserendo il gettone in una colonna casuale. La duplicazione del server avviene con la funzione `fork`, mentre l'esecuzione di un caso particolare di client con la funzione `exec1`. Il caso particolare di client viene eseguito specificando il carattere `~` invece di `*`.